



UNIVERSITÀ DI PISA

Master Of Science in Computer Engineering Electronics and Communications Systems Matrix Multiplier

Group Members:

Megan Maremmanni

Eleni Zewdu Belayhun

Academic Year: 2024/2025

SUMMARY

INTRODUCTION.....	3
GENERAL INTRODUCTION.....	3
GENERAL PROBLEM DESCRIPTION	3
PROJECT PROBLEM DESCRIPTION.....	4
IMPLEMENTATION	5
INTERFACE	5
BLOCK SCHEME	6
GENERAL STRUCTURE	7
VERIFICATION	10
TEST PLAN – TEST BENCH.....	10
SYNTHESIS.....	12
TIMING REPORT	13
MOST CRITICAL PATH	14
POWER CONSUMPTION REPORT	15
RESOURCE UTILIZATION REPORT	15
CONCLUSION	16

INTRODUCTION

GENERAL INTRODUCTION

Matrix Multiplication is an extremely useful and fundamental operation in various scientific fields.

It involves computing the dot product of rows and columns, which requires a series of multiplications and additions. While conceptually simple, matrix multiplication becomes computationally intensive as matrix dimensions grow, requiring significant processing power and memory bandwidth.

It is widely used in real world applications, such as Wireless communications, for example MIMO (Multiple Input Multiple Output) systems for signals processing and in machine learning, it forms the basis of the forward and backward propagation processes in neural networks.

GENERAL PROBLEM DESCRIPTION

Given two matrixes, let $A = [a_{nm}]$ and let $B = [b_{mk}]$ and the result being $C = [c_{nk}]$, we can write the multiplication as follows:

$$c_{nk} = \sum_{j=1}^n a_{nj} \cdot b_{jk}$$

Extended: $c_{nk} = a_{n1} \cdot b_{1k} + a_{n2} \cdot b_{2k} + \dots + a_{nm} \cdot b_{mk}$

The Matrices are the ones that follow:

$$A = \begin{bmatrix} a_{11} & \dots & a_{1m} \\ \vdots & \ddots & \vdots \\ a_{n1} & \dots & a_{nm} \end{bmatrix} \quad B = \begin{bmatrix} b_{11} & \dots & b_{1m} \\ \vdots & \ddots & \vdots \\ b_{k1} & \dots & b_{mk} \end{bmatrix} \quad C = \begin{bmatrix} c_{11} & \dots & c_{1n} \\ \vdots & \ddots & \vdots \\ c_{k1} & \dots & c_{nk} \end{bmatrix}$$

We have A and B as input matrices and C is the result of the computation between A and B.

PROJECT PROBLEM DESCRIPTION

The request to design a digital circuit that implements a matrix multiplication:

- Matrix A is an $N \times M$ matrix.
- Matrix B is an $M \times K$ matrix.
- with N , M , and K integers with maximum value of MAX_SIZE (a generic parameter of the circuit).
- The resulting matrix is named C .

The interface of the circuit to be designed is as follows in Figure 1:

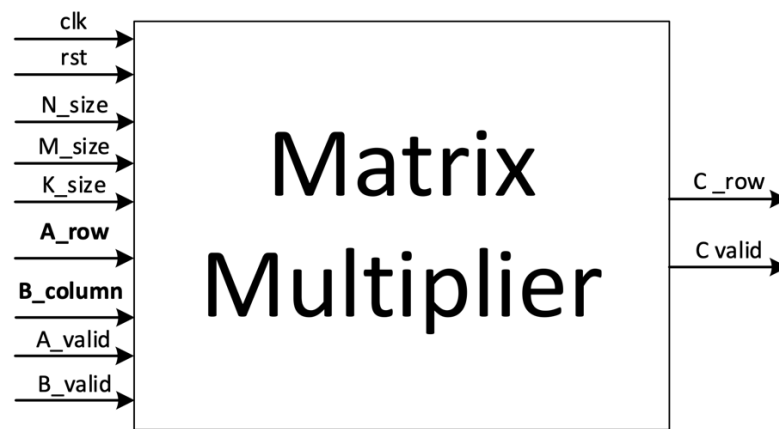


Figure 1: Matrix Multiplier block.

- N , M , K_size signals indicate the value of N , M , K ; their bit width is related to the generic MAX_SIZE .
- A_row and B_column , are used to input the matrices A and B row by row and column by column respectively, in combination with the respective $\{A, B\}_valid$ signals.
- Please note that the value of N , M , and K may change from one operation to another, but is anyway limited to MAX_SIZE .

The resulting matrix C , which of course will have the dimension $N \times K$, is outputted row by row.

IMPLEMENTATION

INTERFACE

The interface is the following:

Inputs:

- clk : Clock signal
- rst : Reset signal (active high)
- N_size : Number of rows of Matrices A and C
- M_size : Number of columns for A and number of rows for B
- K_size : Number of columns for Matrices B and C
- A_row : Input matrix A stored row-wise
- B_column : Input matrix B stored column-wise
- A_valid : Valid signal for A input (rows)
- B_valid : Valid signal for B input (columns)

Outputs:

- C_row : Output matrix C stored row-wise
- C_valid : Output valid signal for C

BLOCK SCHEME

The code is implemented as shown in Figure 2.

- We implemented the design by using two Look up tables(LUTs) to store the values of Matrix A and Matrix B coming as inputs.
- At the end of the multiplication we store the result, row by row, in another LUT and then the final output is shown.

There is a wrapper that wraps the module between two layers of registers.

- This structure is necessary since Vivado only performs timing analysis on Register-Logic-Register paths

The block scheme of the architecture is implemented as follows:

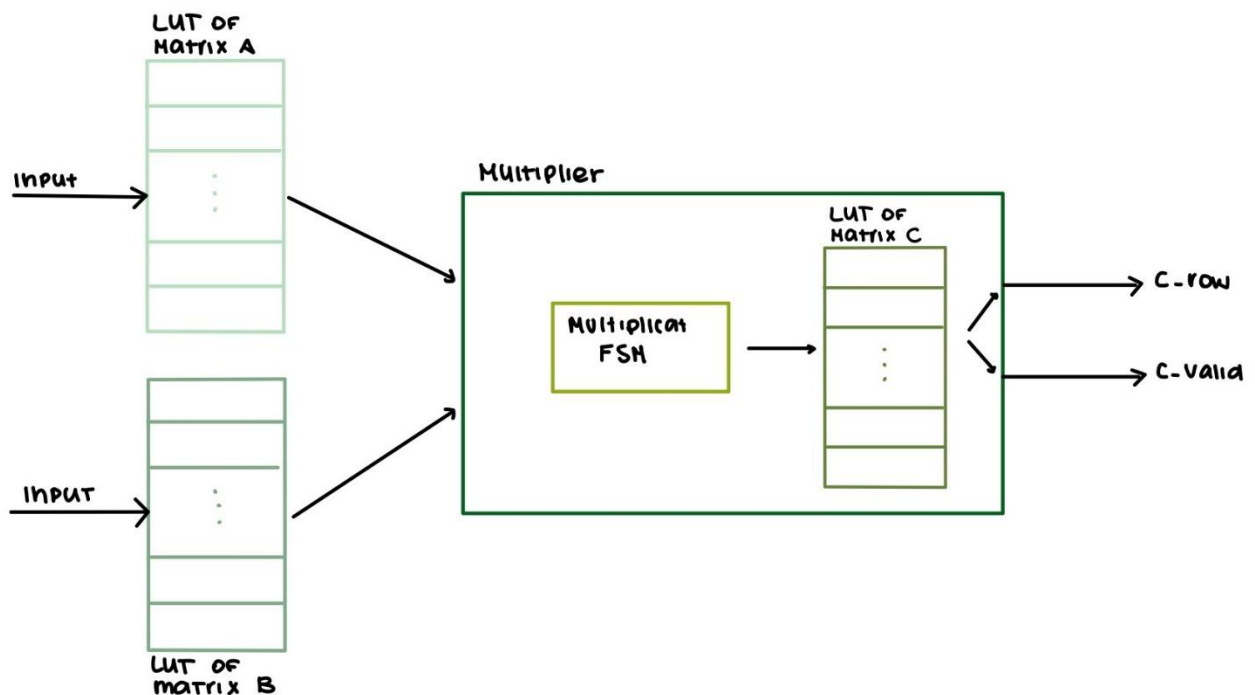


Figure 2: Block diagram

GENERAL STRUCTURE:

Signals.vhdl

To improve the code's readability and modularity, a custom signals package is included to reference the system's inputs and outputs in a clean and structured way.

The types and subtypes are declared as follows in the code:

```
-----
-- VHDL package for matrix multiplication
-- This package defines the data types that we used in the matrix
multiplication design
-----

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

package signals is

-- Define the dimensions for our matrices
constant MAX_SIZE      : integer := 3;
constant WORD_LENGTH   : integer := 3; -- we chose 3 bits for our word length,
because it is a good trade-off

--define our types to ease the declaration
subtype word_bit is std_logic_vector(WORD_LENGTH - 1 downto 0);
-- The dimension of out_bit is chosen as 2*WORD_LENGTH + MAX_SIZE - 1 to
accommodate:
-- 1. The result of multiplying two WORD_LENGTH-bit numbers, which can be up to
2*WORD_LENGTH bits wide.
-- 2. The summation of up to MAX_SIZE partial products during matrix
multiplication, requiring additional bits.
-- This ensures no overflow occurs and the output can store the largest
possible result.
subtype out_bit is std_logic_vector(2*WORD_LENGTH + MAX_SIZE -1 downto 0);

-- Define the array of vectors to ease the code readability
type input is array (MAX_SIZE - 1 downto 0) of word_bit; -- array for inputs
type output_c is array ( MAX_SIZE - 1 downto 0) of out_bit; -- array for output
c_row

end package signals;
```

Matrix_LUT_A

This is the first Look-Up Table (LUT), designated for storing the input data corresponding to Matrix A. The data is loaded sequentially, element by element.

Matrix A is stored in a row-wise format to align with the requirements of the matrix multiplication logic, once the data is valid.

To maintain synchronization with the second LUT and the overall system architecture, the design ensures that the data loading phase is fully completed before going into the computation stage (multiplication stage).

Matrix_LUT_B

This is the second Look-Up Table (LUT) used to store incoming data for Matrix B. The data is loaded sequentially, element by element, once the data is valid.

Matrix B is stored in a column-wise format to align with the requirements of the matrix multiplication logic.

To ensure proper synchronization with the first LUT and the overall system, the design waits for the data loading process to complete before initiating the multiplication phase.

This approach guarantees data consistency and enables a more efficient implementation of the matrix multiplication operation.

Matrix_LUT_C

A third Look-Up Table (LUT) is utilized to store the output values resulting from the multiplication of Matrix A and Matrix B.

Each computed element of the resulting matrix is written into a specific address of this LUT, corresponding to its position in the final matrix.

Once stored, each output value is mapped to C_row, marking the completion of the multiplication process, with the C_valid signal stating the validity of the results.

The output data is organized and retrieved in a row-wise manner, aligning with the structure of the resultant matrix.

MatrixMultiplier

This is the main body of the implementation, where the architecture of the system is defined in accordance with the problem specifications: three Look-Up Tables (LUTs) are used for storing input and output data and they are instantiated as components.

All internal signals are declared early in the architecture to support proper synchronization and value assignment throughout the design and the signal package is included: the component instantiation is completed through explicit port mapping, establishing all necessary interconnections.

The main functionality of the design is implemented as a Finite State Machine (FSM), which provides a structured and efficient approach for managing the matrix multiplication process.

The `state_reg` process handles synchronous state transitions and updates counters based on the clock and reset signals. It resets all states and counters when the reset signal is active and transitions the current state (`curr_state`) to the next state (`next_state`).

The FSM process implements the combinational logic for the finite state machine, determining the next state and control signals based on the current state and input conditions. It manages, for example, the enabling of LUTs, starting of multiplication, and writing to the result matrix.

The `MUL` process performs the core matrix multiplication by calculating the dot product of corresponding elements from matrices A and B. It resets accumulators when the reset signal is active and accumulates the results into the `dot_product` signal during multiplication.

The `RESULT_OUT` process gives the output matrix row by row using LUT C. Once there is the reset condition, it clears the output row and resets the internal counters. It sequentially assembles each row and signals its readiness with the `C_valid_signal`.

We defined the states:

1. **IDLE**: the base state where everything is ready and initialized to start the operations;
2. **LOAD_MAT**: state where the loading of the look up tables happens for both matrices. Once we finish loading the look up tables we can go to the next state;
3. **CHECK**: extra state added to check if all the operations, before the multiplication, have been completed and everything is ready. If so, we go to the next state;
4. **MULTIPLY**: state where multiplication operation happens;
5. **MAT_C**: state where there is the storing of the result in the LUT;
6. **RESULT**: state that prepares the final output matrix C by reading elements from LUT C;
7. **END_MUL**: this state signals the completion of the matrix multiplication process.

VERIFICATION

TEST PLAN – TEST BENCH

The verification of the design was performed using a VHDL testbench, simulated with ModelSim-Intel FPGA Standard Edition.

The testbench is configured to validate the functionality of the matrix multiplication module, given row and column of matrices as inputs.

For testing purposes, a 3×3 matrix configuration was selected, as it provides an effective balance between simplicity and comprehensiveness, without excessive simulation complexity.

Additionally, a dedicated control flag was implemented to automatically terminate the simulation when the matrix multiplication process ends, ensuring efficient simulation.

A selection of significant events of the simulations are reported in this documentation.

First, in Figure 3, there is the Loading of one of the LUTs (Matrix_LUT_A):

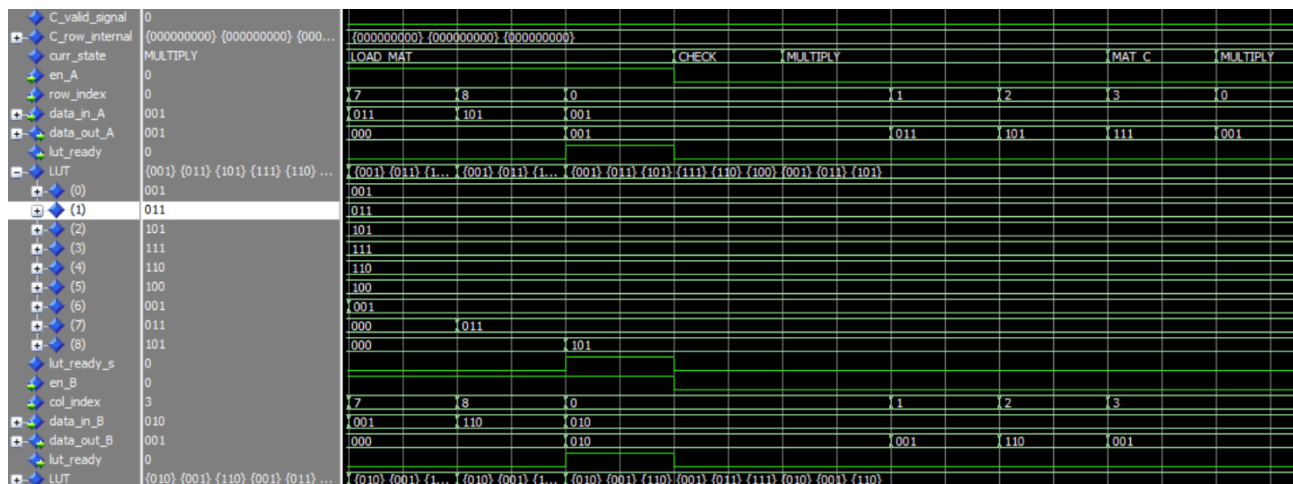


Figure 3: Loading of one LUT with values

After the inputs are valid, the loading happens and once it's ready the multiplication process starts and the correct results are stored in the LUT C for the output, as shown in figure 4:

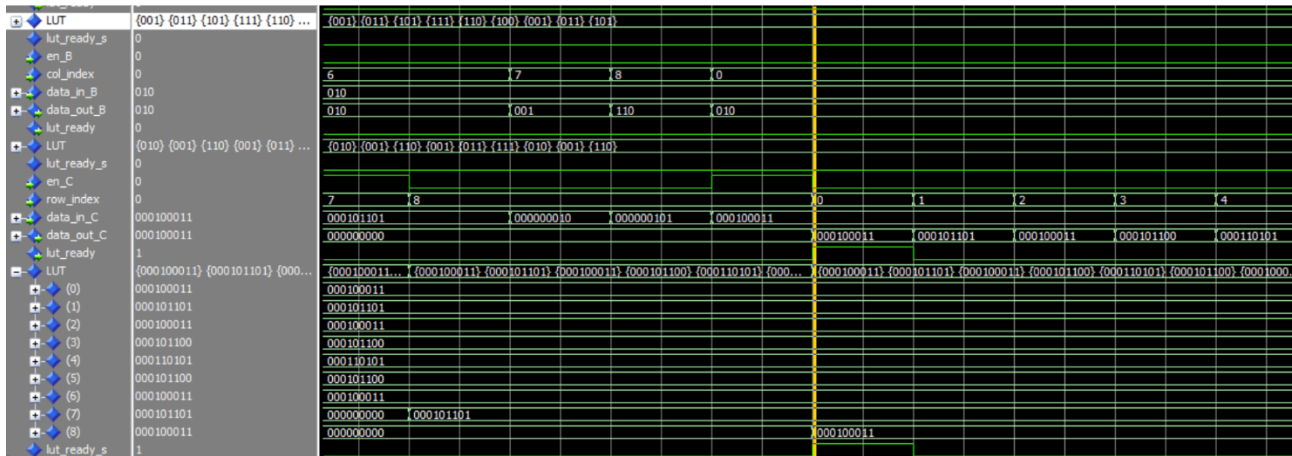


Figure 4: LUT C result storage

After the LUT C is loaded, the output is given to C_row and C_valid becomes 1 as soon as there is the valid row, as shown in Figure 5:

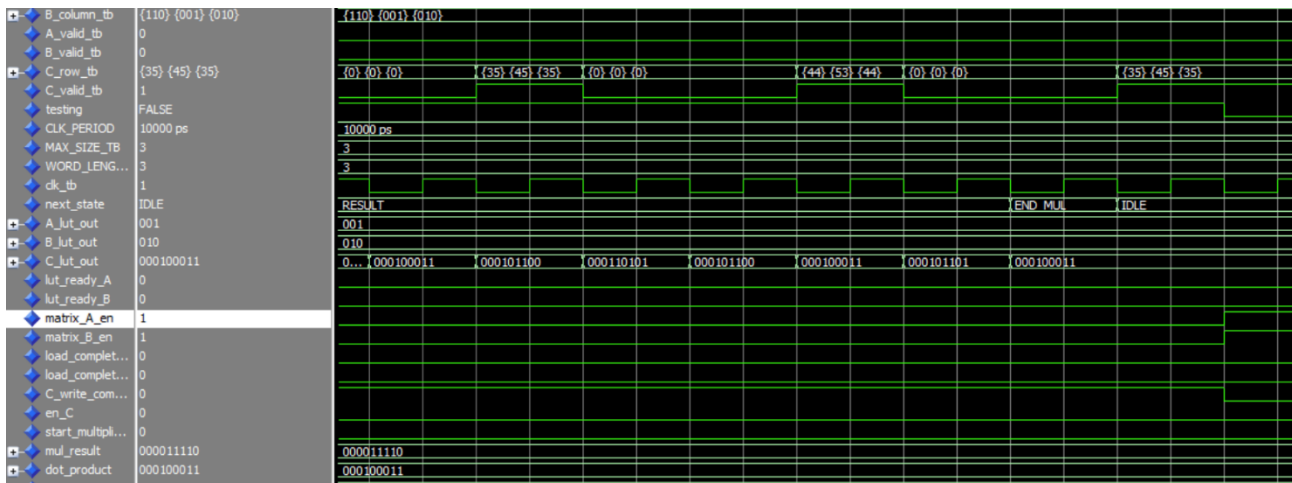


Figure 5: output of the results

SYNTHESIS

The synthesis of the code was done in Vivado simulation tool, using the Zybo development board xc7z010clg400-1. (Figure 6)

The synthesis was done without mapping the actual physical I/O ports of the FPGA pins, using the -mode out_of_context option.

A timing constraint was set at 125MHz , to obtain a timing report.

Some warnings came up during simulation:

- This is because we used the out of context mode, so the accuracy of the timing is affected, however this does not impact the validity of the current results.

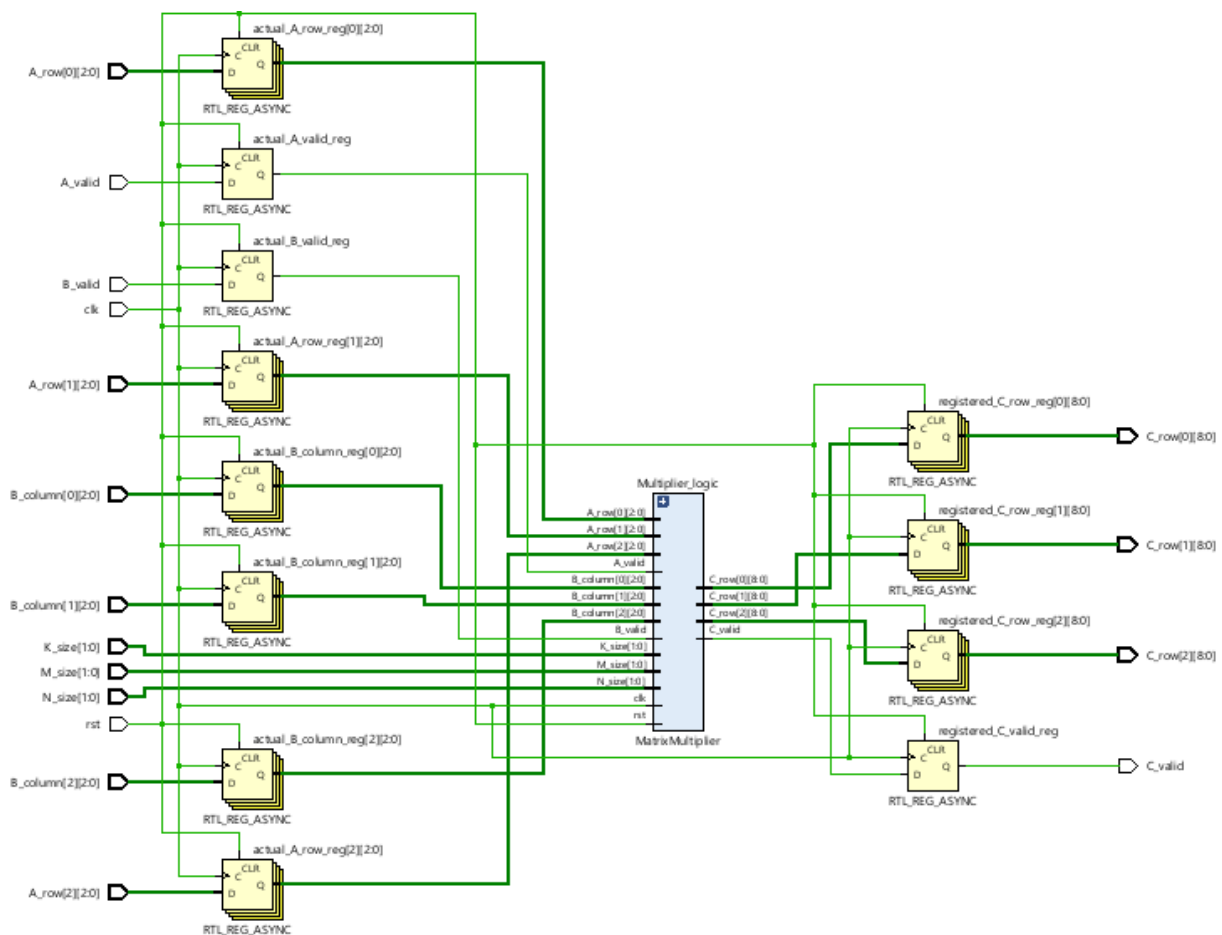


Figure 6: Vivado RTL Schematic of the circuit

TIMING REPORT

The design timing summary confirms that all user-specified timing constraints for the matrix multiplier implementation have been successfully met as shown in Figure 7.

The analysis shows a worst negative slack (WNS) of 1.484 ns, indicating that the clock period could be reduced by this amount before a setup time violation occurs. Similarly, the hold analysis reports a worst hold slack (WHS) of 0.179 ns.

Additionally, the pulse width check has a worst pulse width slack (WPWS) of 4.500 ns, confirming that the clock pulses are sufficiently wide for reliable register operation.

Design Timing Summary			
Setup		Hold	Pulse Width
Worst Negative Slack (WNS): 1.484 ns		Worst Hold Slack (WHS): 0.179 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns		Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 365		Total Number of Endpoints: 365	Total Number of Endpoints: 249
All user specified timing constraints are met.			

Figure 7: Vivado Timing Report

MOST CRITICAL PATH

The timing analysis of the matrix multiplier design, as shown in figure 8, highlights the most critical path. The most critical timing path is "Path 1", exhibiting slack value of 1.484 ns.

This path starts from the actual_B_valid_reg flip-flop and traverses through multiple stages of combinational logic, including LUT6 and LUT5 elements responsible for computing the next state (curr_state) and updating the row counter (row_counter).

Name	Slack ^1	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement
Path 1	1.484	3	19	actual_B_valid_reg/C	Multiplier_lo...ter_reg[0]/CE	3.230	1.031	2.199	5.0
Path 2	1.484	3	19	actual_B_valid_reg/C	Multiplier_lo...ter_reg[1]/CE	3.230	1.031	2.199	5.0
Path 3	1.625	3	19	actual_B_valid_reg/C	Multiplier_lo...ter_reg[2]/CE	3.089	1.031	2.058	5.0
Path 4	1.625	3	19	actual_B_valid_reg/C	Multiplier_lo...ter_reg[3]/CE	3.089	1.031	2.058	5.0
Path 5	1.662	3	19	actual_B_valid_reg/C	Multiplier_lo...ter_reg[0]/CE	3.090	1.031	2.059	5.0
Path 6	1.662	3	19	actual_B_valid_reg/C	Multiplier_lo...ter_reg[1]/CE	3.090	1.031	2.059	5.0
Path 7	1.662	3	19	actual_B_valid_reg/C	Multiplier_lo...ter_reg[2]/CE	3.090	1.031	2.059	5.0
Path 8	1.662	3	19	actual_B_valid_reg/C	Multiplier_lo...ter_reg[3]/CE	3.090	1.031	2.059	5.0
Path 9	2.063	3	19	actual_B_valid_reg/C	Multiplier_lo...ter_reg[0]/D	2.885	1.031	1.854	5.0
Path 10	2.165	2	19	actual_B_valid_reg/C	Multiplier_lo...ate_reg[0]/D	2.499	0.699	1.800	5.0

Figure 8: Paths of the circuit

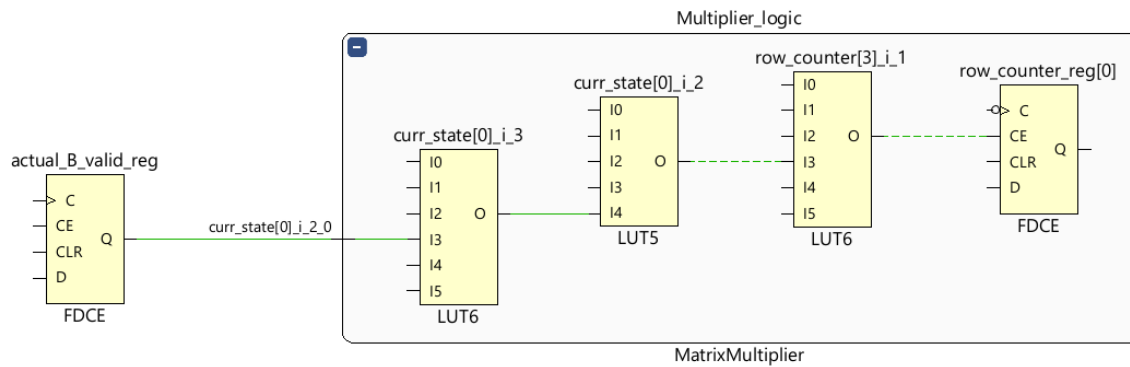


Figure 9: Path 1 critical path

POWER CONSUMPTION REPORT

Figure 10 shows the power analysis of the matrix multiplier design.

Most of this power, approximately 95% (0.090 W), is attributed to static device power, which is inherent to the FPGA hardware itself regardless of activity.

The remaining 5% (0.004 W) corresponds to dynamic power, which is associated with the switching activity of the design during operation.

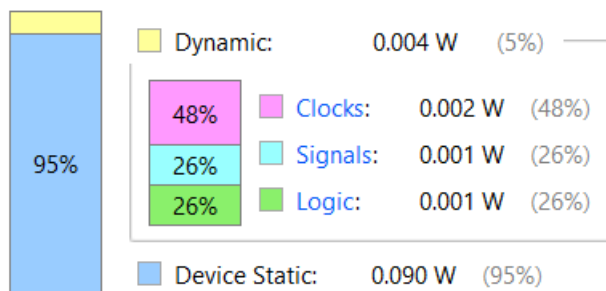


Figure 10: Power Consumption Report

RESOURCE UTILIZATION REPORT

Figure 11 presents the resource utilization report for the Matrix Multiplier.

The implementation utilizes only 236 Look-Up Tables (LUTs), resulting in a utilization rate of 1.34% and it consumes 249 Flip-Flops (FFs), corresponding to a 0.71% utilization.

Such low resource usage suggests that the design avoids excessive combinational complexity and makes effective use of sequential logic through pipelining and finite state machines.

The matrix multiplier can thus be easily scaled or incorporated into more complex systems without placing significant demands on the hardware, making it a robust and adaptable solution for FPGA-based applications.

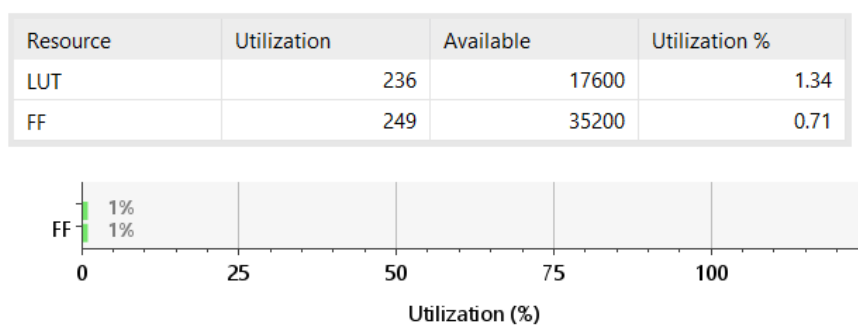


Figure 11: Resource Utilization Report

CONCLUSION

The primary objective of this project was to develop a Matrix Multiplier using VHDL, with the goal of implementing it on a digital circuit targeting an FPGA board.

The designed circuit performs matrix multiplication by processing the rows and columns of two matrices, stored in 2 different Look up tables (LUT), executing the necessary multiplications and intermediate sums, and storing the results in a third Lookup Table (LUT), which subsequently provides the output.

Additionally, there is a wrapper module that was implemented to encapsulate the core logic between layers of registers, establishing register-logic-register paths. This structure is essential for analysing the system's behaviour and performance.

The functionality of the code was validated using a VHDL testbench, with a 3x3 input matrix. Parameters were set with a MAX_SIZE of 3 (indicating the maximum matrix dimensions) and a WORD_LENGTH of 3 bits, chosen to balance simplicity with computational efficiency.

Looking ahead, the design can be scaled to support larger and more diverse matrix types, broadening its applicability for more complex digital systems.